

that frequently occur. One is the case of memory leaks, wherein the programmer fails to release the dynamically allocated memory. The other is when the programmer uses the memory after having released it, known as a 'Use after free' error. In managed environments like Java, which have automatic garbage collection support, the programmer does not face the burden of explicitly having to free dynamically allocated memory after its use is over. However, statically compiled languages like C/C++, which do not have automatic garbage collection support, require the programmer to explicitly free the dynamically allocated memory. It is important to correctly remember to free the memory after its use, while at the same time, you should not end up freeing it too early, lest you run into a 'Use after free' error. Consider the code snippet given below:

```
void foo(int a);
{
    int* p = malloc(sizeof(int) * a);
    assert (p != null);
    *p = a;
    int result = bar();
    if (result == 0)
        printf("result was zero\n");
    else
    {
        free(p);
    }
    printf("the value stored in 'p' is %d\n", *p);
    return;
}
```

In this snippet, the code allocates memory dynamically, and the address is stored in the variable 'p'. Note that there is one path in function 'foo' in which the allocated memory is freed, whereas there is another path in which the programmer has omitted to free the allocated memory. This constitutes a memory leak. On the other hand, consider the *printf* statement just before the return statement in function 'foo'. This is a possible dereference of the pointer 'p' after it has been freed, since if we reach here after passing through the else clause, 'p' would already have been freed—an example of a 'Use after free' error.

Leaks are not limited to dynamically allocated memory, but can occur with any system resource such as file handles, file descriptors, locks, etc. For instance, the programmer may open a file using a file descriptor, but forget to close the file using the file descriptor. This would result in a file descriptor leak. Similarly, it is possible for the programmer to acquire a lock, but forget to release it once the critical section is done. Failure to release a lock can even lead to a deadlock. Therefore, programmers need to ensure that the resources they acquire are released as soon as their use is over, and no sooner than that. A

release done too early, or too late (or perhaps never) can lead to either correctness issues, and/or performance problems.

There are two other important classes of memory access errors—namely, 'Uninitialised memory reads', and out-of-bounds access. Consider the following code snippet:

```
initialise(int* p, int size, int init_val)
{
    for (int i = 0; i <= size; i++)
        *(p+i) = init_val;
}
```

Can you figure out the coding error in the above code snippet? Well, the programmer has made a mistake in specifying the terminating condition for the 'for' loop. Assume that the variable 'p' points to an array of size 10. The calling code passed the argument 'p' and the size as 10. However, the 'for' loop in the function 'initialise' actually tries to initialise 11 elements instead of 10, and performs an out-of-bounds access when it tries to initialise $*(p+10)$. While the above code snippet is trivial enough for the error to be detected by visual inspection, such out-of-bounds accesses in real-life applications are extremely difficult to find. Tools that can detect such errors need to perform sophisticated inter-procedural analysis to detect these types of errors.

While out-of-bounds errors are typically caused by an off-by-one error in loop bound calculation, another major class of memory access error is 'Uninitialised memory read'. Consider the code snippet below:

```
int foo(int size)
{
    int* p = (int*) malloc(sizeof(int) * size);
    assert(p != 0);
    int result = bar(p);
    return result;
}

int bar(int* p)
{
    int my_val = *p;
    //do something with my_val
}
```

Note that while the memory pointed to by the variable 'p' is allocated in the function 'foo', it was not initialised in 'foo'. However, the code in the function 'bar' tries to read from the memory pointed to by 'p'. This is an 'Uninitialised memory read' (UMR). Detecting UMR errors requires tracking when a memory location is initialised. Detecting UMR errors through static analysis is difficult, and hence there are runtime detection tools to spot memory access errors like UMR. In next month's column, we will discuss the various static and runtime analysis tools to detect software bugs.